

Data Structure

11. 그래프, DFS & BFS

11주차: Graph, DFS, BFS

목차 INDEX

01 그래프

- Vertex & Edge
- Tree / Graph
- Directed & Undirected
- Weighted & Unweighted

02 DFS

- DFS
- Recursion / Call Stack
- visited
- Cycle

03 BFS

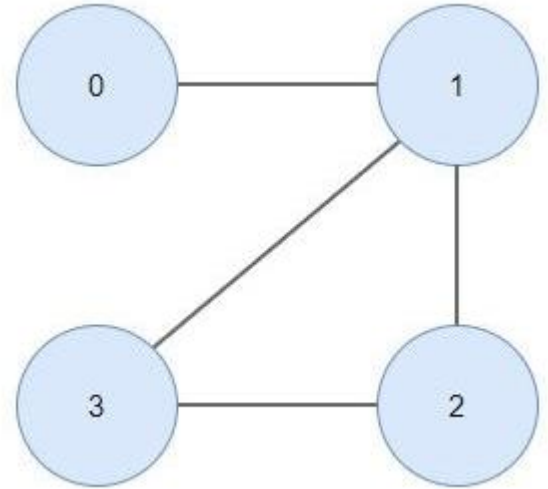
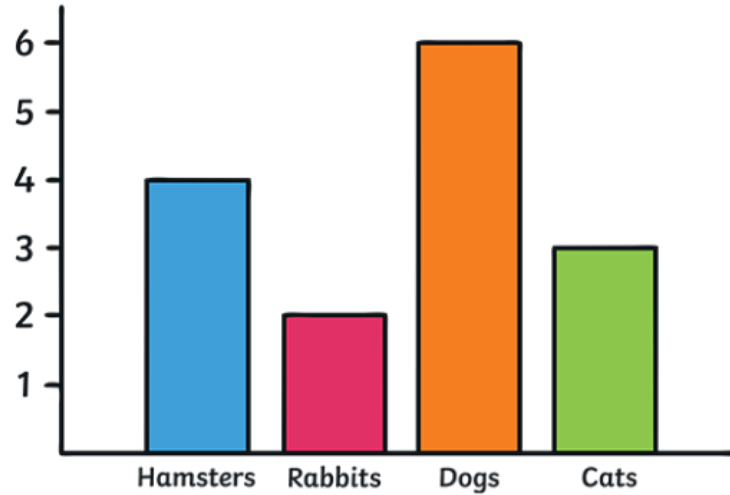
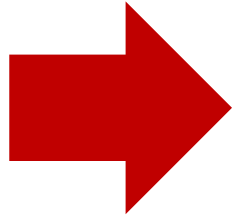
- BFS
- Queue
- visited

01

그래프

Graph

그래프



γράφειν

고대 그리스어

(코이네 그리스어)

쓰다 / 그리다 / 새기다

Graphium

라틴어

필기구

Graphe

프랑스어

글자 / 기호

Graph

영어

그래프

(대상 간의 관계를 표현하는 그림)

그래프

SNS

사람 = Node
친구 = Edge

지하철

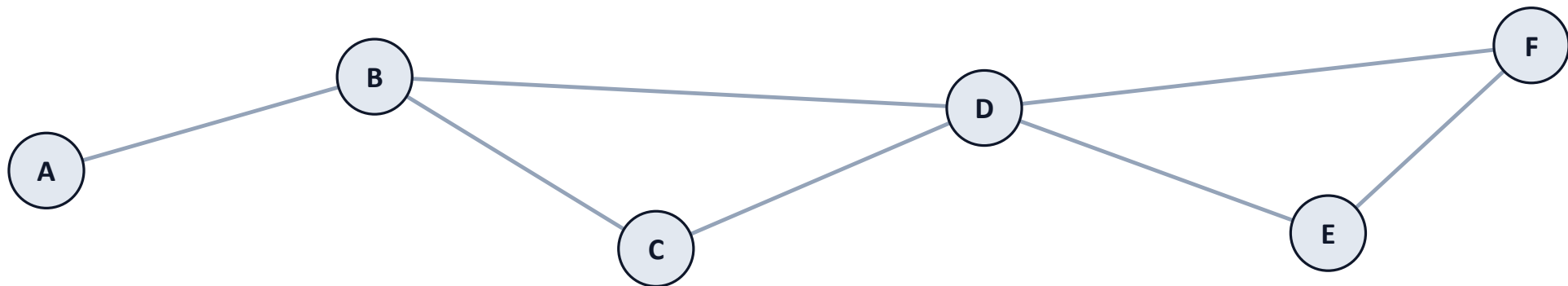
역 = Node
노선 = Edge

도로

교차로 = Node
도로 = Edge

웹

페이지 = Node
링크 = Edge



복잡한 관계를 추상화하는 기법

그래프의 용어는 트리와 공유함 (트리 $\subset$ 그래프)

Vertex & Edge

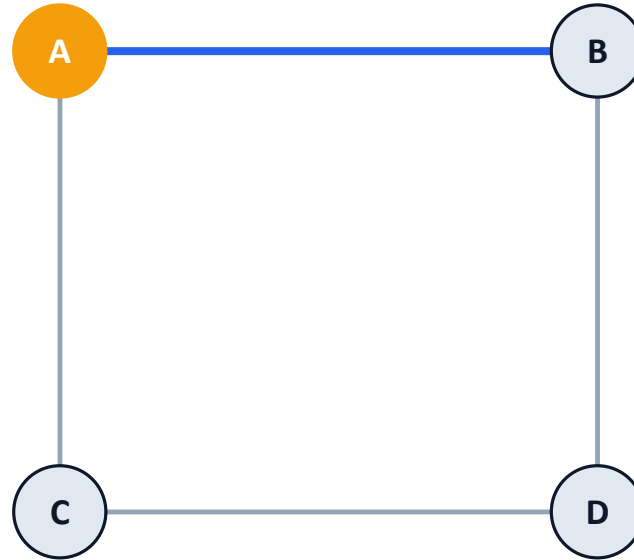
$$G = (V, E)$$

Vertex / Node

대상 자체: 사람, 역, 페이지, 상태

Edge

두 Vertex 사이의 관계



$$V = \{A, B, C, D\}$$

$$E = \{(A,B), (A,C), (B,D), (C,D)\}$$

Directed & Undirected

UNDIRECTED

친구 관계



$A - B$: 서로 연결

DIRECTED

팔로우 관계



$A \rightarrow B \neq B \rightarrow A$

Weighted & Unweighted

UNWEIGHTED



A — B : 서로 연결

WEIGHTED



A — B : 사이에 가중치가 있음

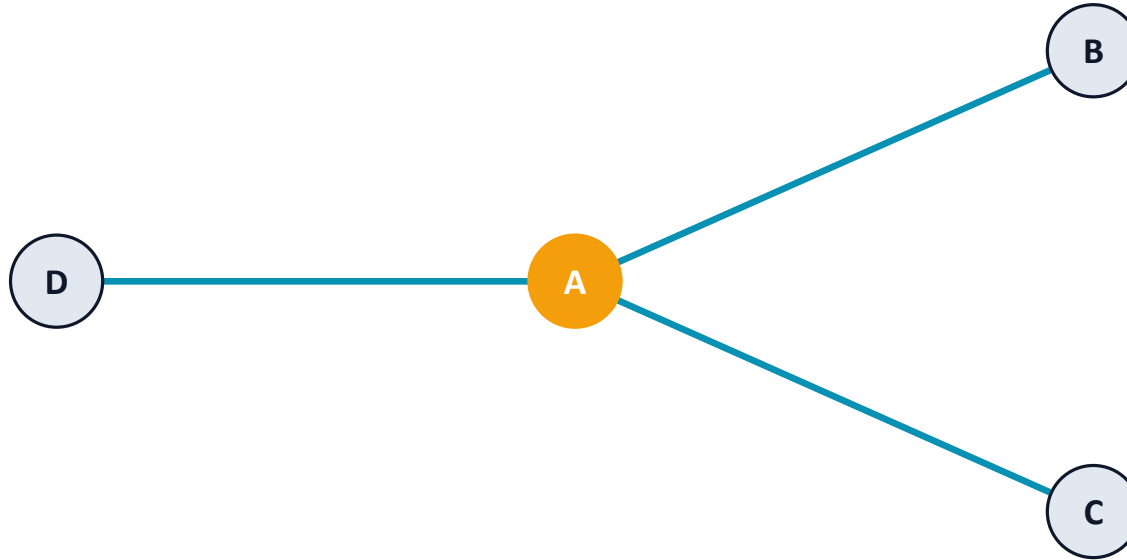
Degree: 한 노드에 연결된 엣지 수

Undirected

Degree = 연결된 Edge 수

Directed

In-degree
Out-degree



A: Undirected Graph

Degree = 3

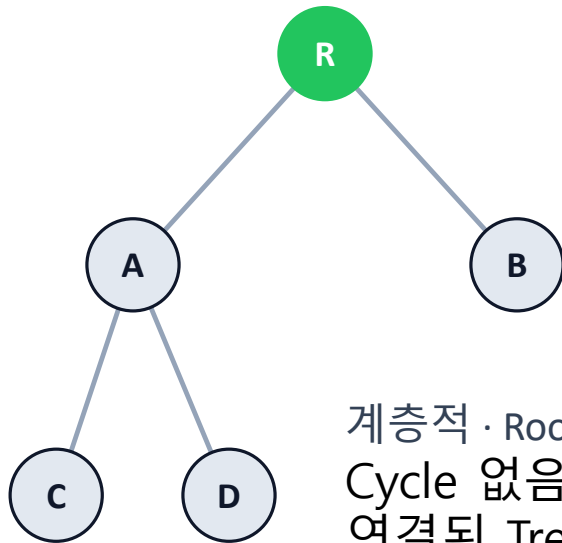
A: Directed Graph

In-degree = 1

Out-degree = 2

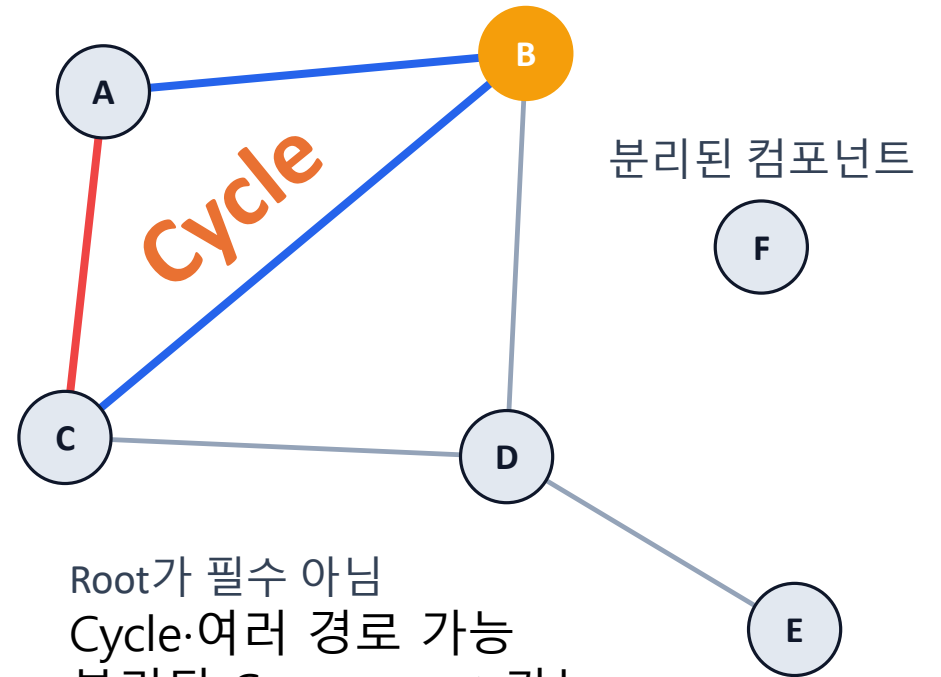
트리와 그래프의 관계

TREE



계층적 · Root 관점
Cycle 없음
연결된 Tree: $E = V - 1$

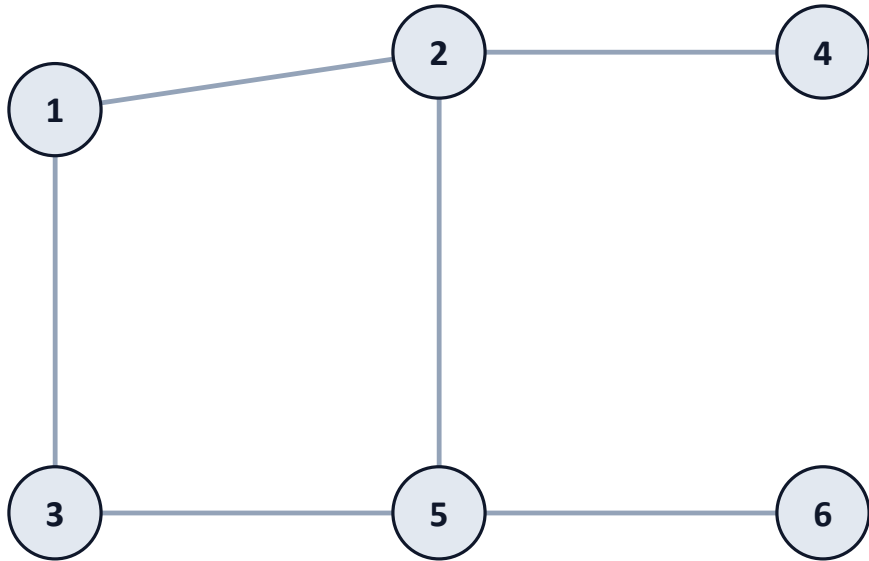
GENERAL GRAPH



Root가 필수 아님
Cycle · 여러 경로 가능
분리된 Component 가능

트리 $\subset$ 그래프

Graph Traversal: DFS & BFS



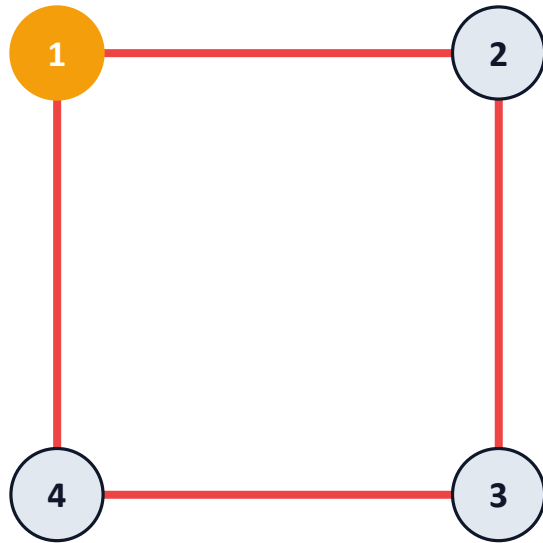
DFS · Depth-First

한 방향으로 갈 수 있는 만큼 깊게

BFS · Breadth-First

시작점에서 가까운 Node부터 Level 단위

Visited



1 → 2 → 3 → 4 → 1 (다시 방문하면 안됨)

```
visited = [False] * (N + 1)
```

```
visited[v] = True
```

위 그래프처럼 사이클이 있는 그래프에서는
visited가 없으면 탐색을 무한히 반복할 수 있음

주로 0벡터 또는 False 벡터로 선언

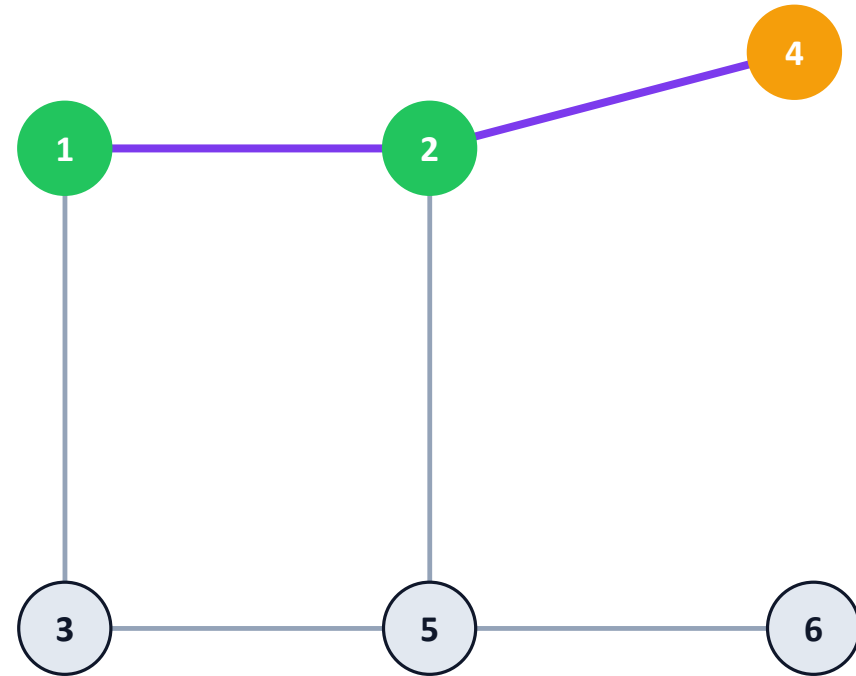
02 DFS

Depth-First Search

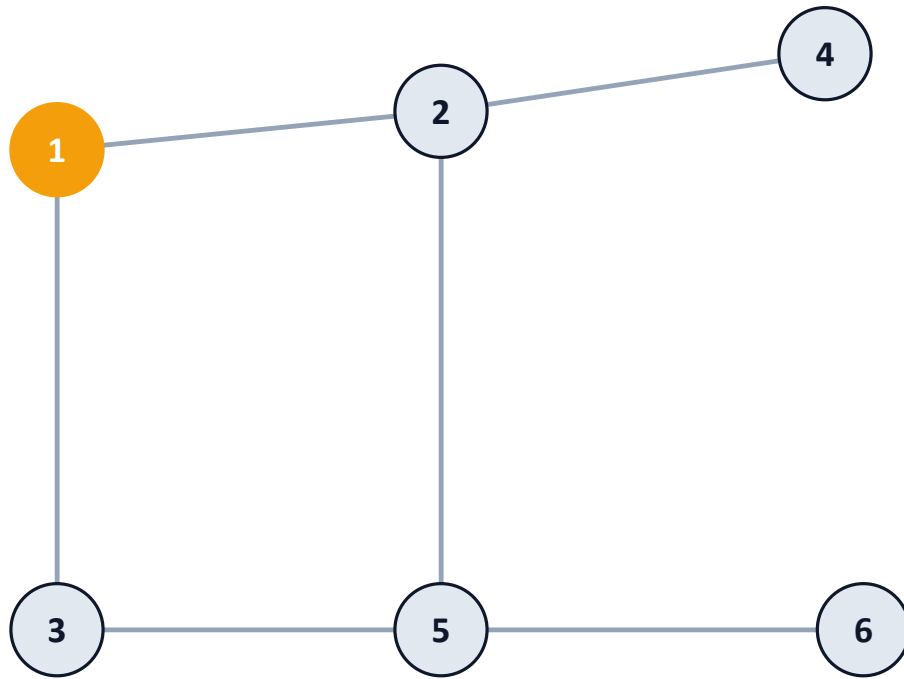
DFS: Depth-First Search

한 방향으로 갈 수 있는 만큼 깊게

더 이상 갈 수 없으면 이전 위치로 돌아와
다른 Neighbor를 탐색한다.



DFS: Depth-First Search

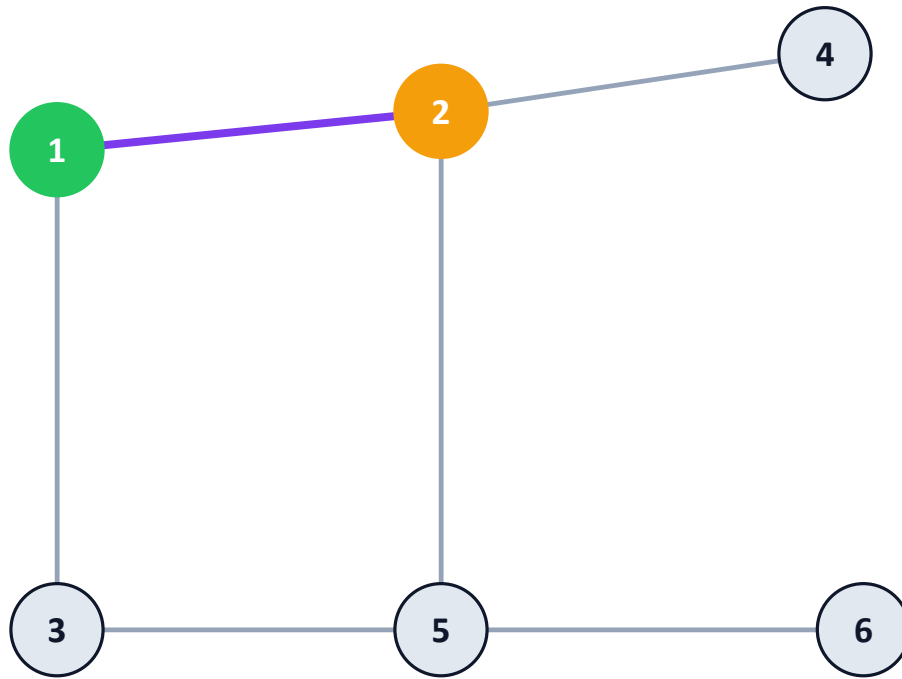


Function Call Stack

Call Stack

[dfs(1)]

DFS: Depth-First Search

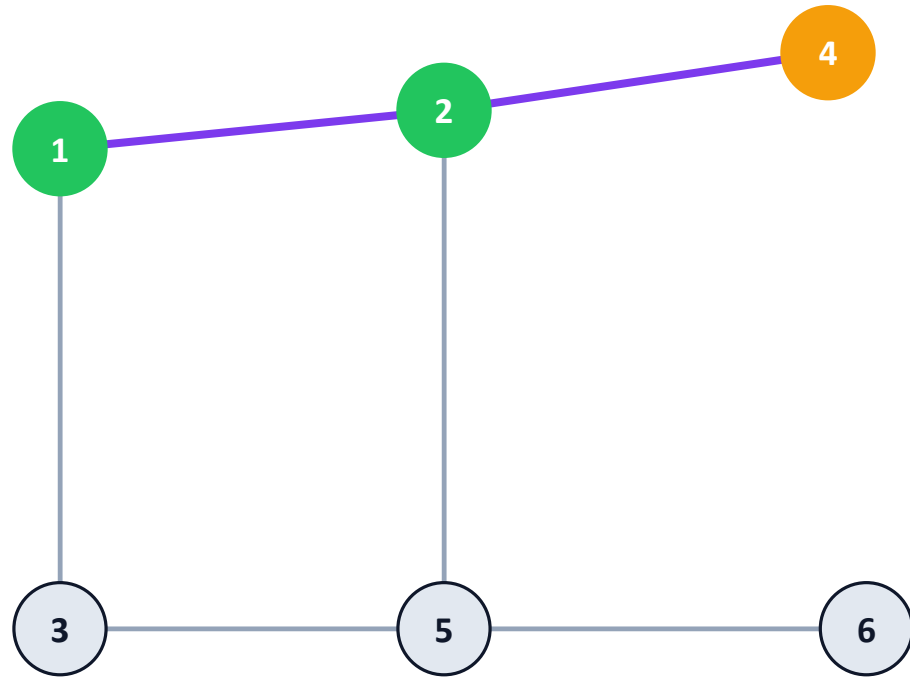


Function Call Stack

Call Stack

```
[ dfs(2) ]  
[ dfs(1) ]
```

DFS: Depth-First Search



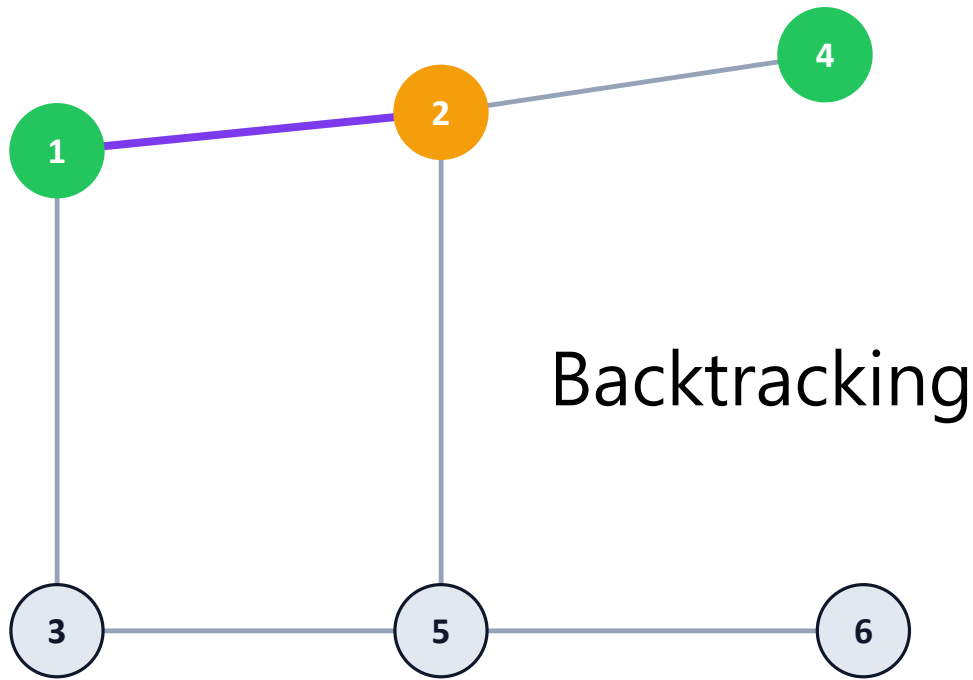
Dead End

Function Call Stack

Call Stack

```
[ dfs(4) ]  
[ dfs(2) ]  
[ dfs(1) ]
```

DFS: Depth-First Search



Function Call Stack

Call Stack

[dfs(2)]

[dfs(1)]

DFS는 LIFO 구조로, 재귀함수 또는 스택을 사용해 구현 가능

DFS: Depth-First Search

PUSH → 깊게

dfs(4)

dfs(2)

dfs(1)

막히면 POP / return

Stack = LIFO

가장 나중에 들어간 호출이 가장 먼저 끝남

DFS의 '깊게 들어갔다가 되돌아오기'와 자연스럽게 대응한다.

The background is a solid dark blue with several geometric shapes. A large, light blue curved shape is in the upper right. A dark blue diagonal band runs from the bottom left towards the top right. A lighter blue diagonal band runs from the bottom right towards the top left.

03

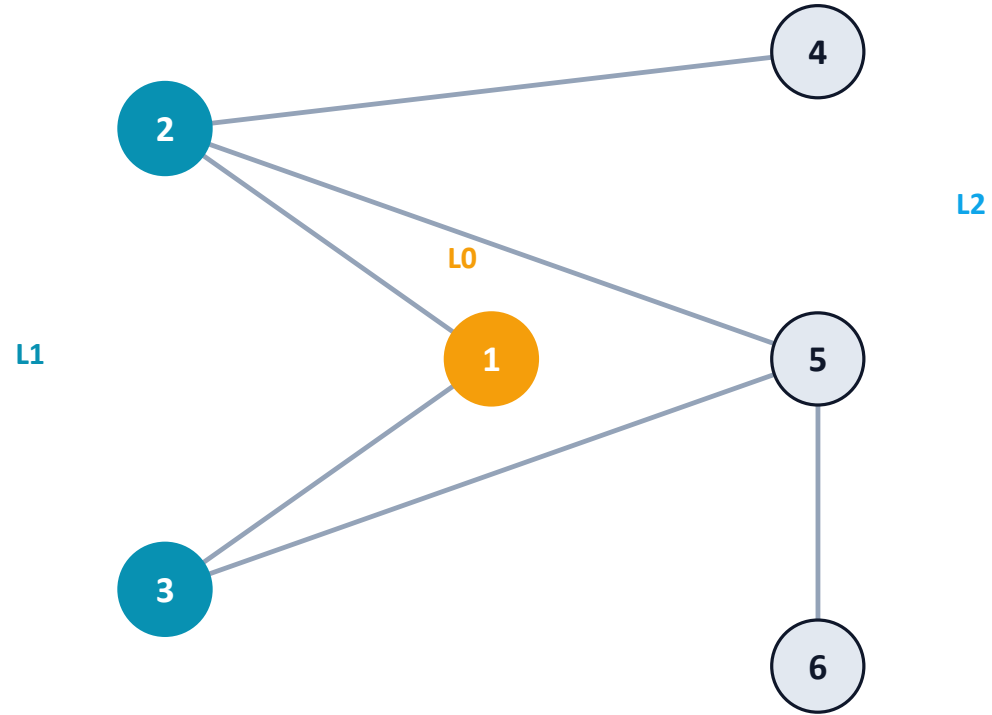
BFS

Breadth-First Search

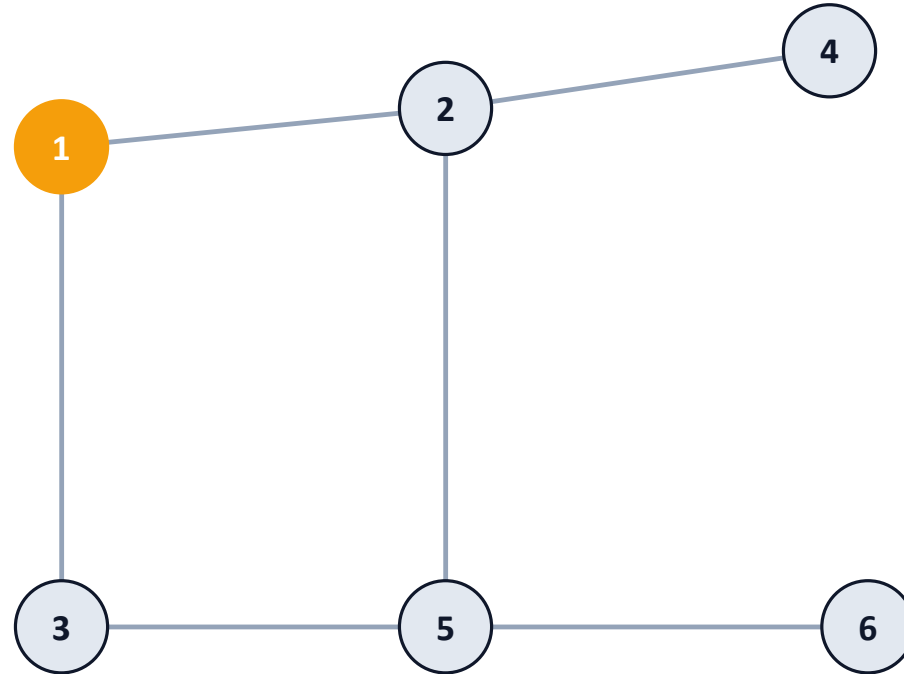
BFS: Breadth-First Search

시작점에서 가까운 Node부터

Level 0 → Level 1 → Level 2
파동처럼 바깥으로 확장



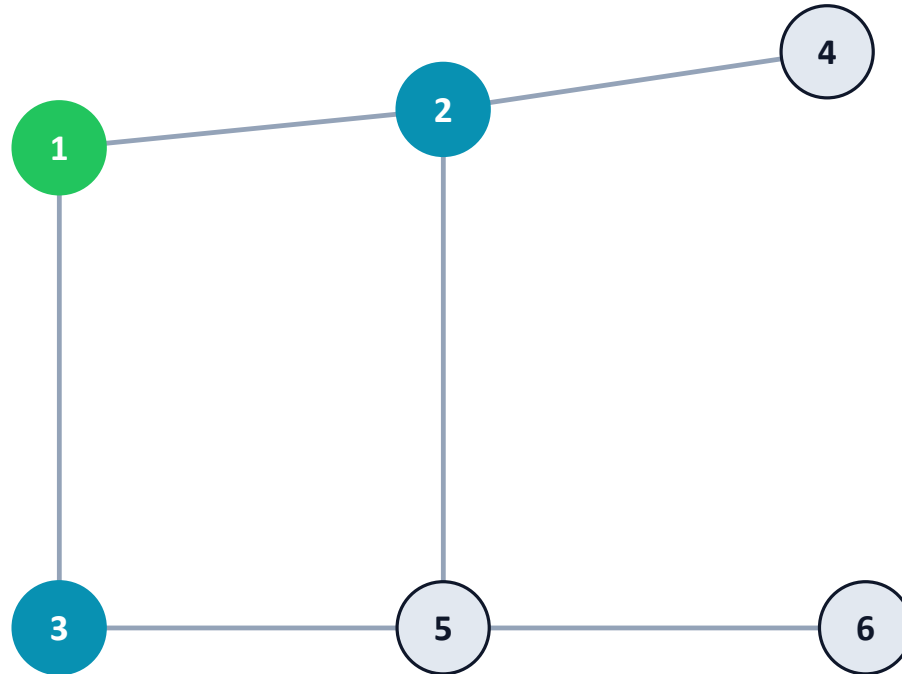
BFS: Breadth-First Search



QUEUE

[1]

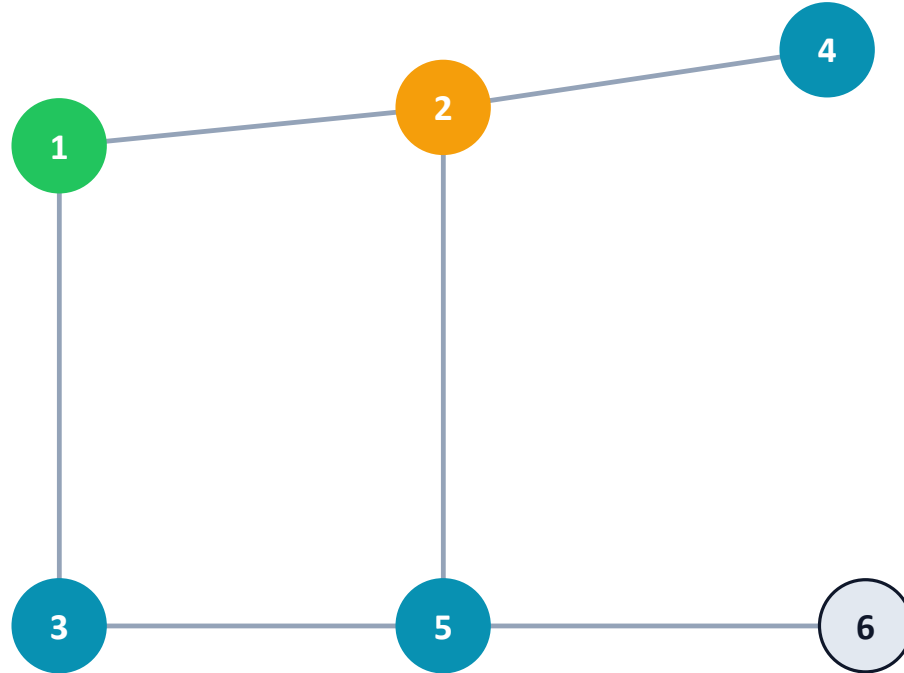
BFS: Breadth-First Search



QUEUE

[2, 3]

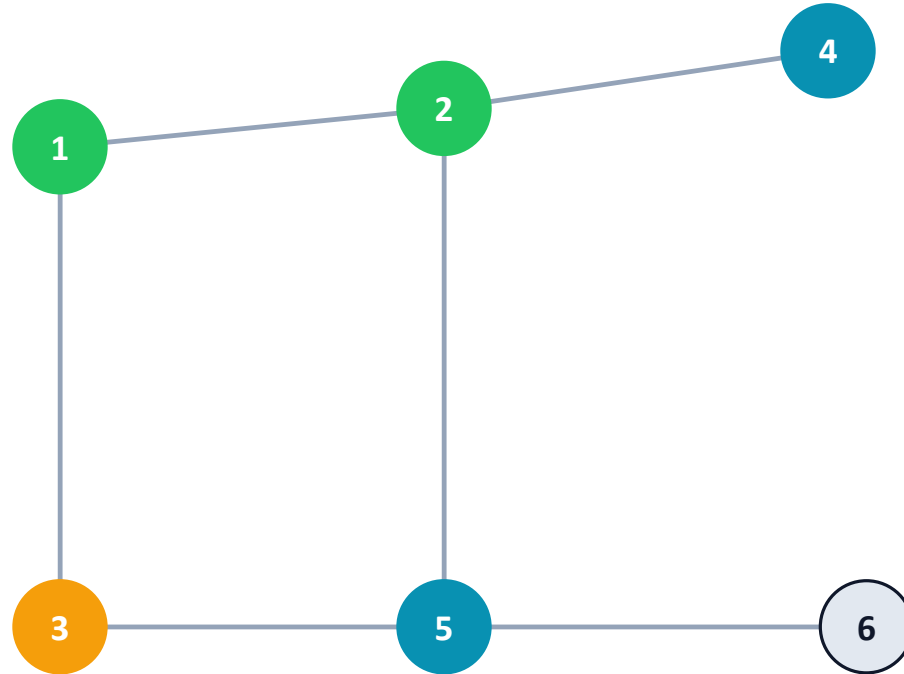
BFS: Breadth-First Search



QUEUE

[3, 4, 5]

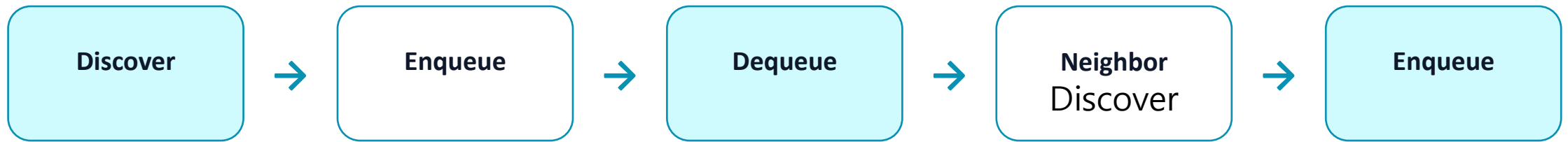
BFS: Breadth-First Search



QUEUE

[4, 5]

BFS: Breadth-First Search



Queue = FIFO

먼저 발견한 Node를 먼저 처리 → 가까운 Level이 먼저 확장